Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

12-1 워드 임베딩

단어의 뜻을 수십~수백 칸의 벡터에 담는 방법을 배웁니다. 원-핫 벡터의 한계에서 출발해 Word2Vec이 "주변 단어 맞히기"로 뜻을 배우는 과정을 식과 숫자로 따라가고, 직접 학습시켜 봅니다.

오늘 다루는 것 (12장 앞부분)

1. NLP에서의 원-핫 인코딩
2. 워드 임베딩: 희소 표현과 밀집 표현
3. 워드투벡터(Word2Vec): CBOW, Skip-gram, 네거티브 샘플링
4. Word2Vec 학습 및 시각화
5. 글로브(GloVe)와 패스트텍스트(FastText)

끝나고 나면 할 수 있는 것

1. 원-핫 벡터의 한계 두 가지와, 임베딩 벡터가 그것을 어떻게 푸는지 설명할 수 있다.
2. CBOW에서 입력이 예측이 되기까지 벡터의 모양이 바뀌는 과정을 따라갈 수 있다.
3. gensim으로 Word2Vec을 학습시키고 비슷한 단어를 찾을 수 있다.

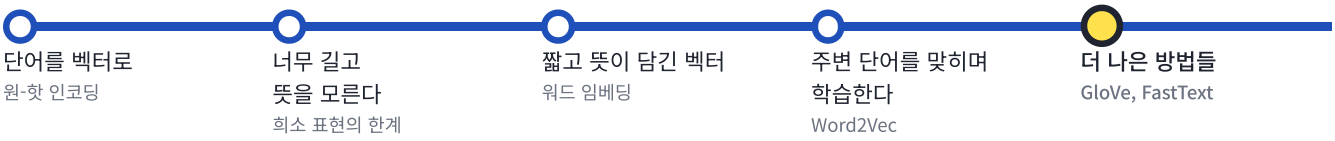
2026년 10월 12일 월요일 14:30 (10/12 자료 2/3)
발표자 설명, 코랩 실습, 퀴즈 돌아가며 풀기 순서로 진행합니다. 코랩에서는 `!pip install konlpy gensim` 을 먼저 실행해 주세요. 실습 5, 6, 8은 학습에 1~3분씩 걸립니다.

12

텍스트 임베딩

11장에서 횟수를 세는 방법의 한계를 봤습니다. Titanic에 물고기 영화가 추천됐고, ship과 boat는 글자가 다르다는 이유로 전혀 다른 단어로 취급됐습니다. 단어마다 번호나 원-핫 벡터를 붙이는 방법에는 단어의 뜻이 들어 있지 않기 때문입니다. 12장은 뜻이 비슷한 단어가 비슷한 벡터를 갖도록 글에서 학습하는 방법, 워드 임베딩을 다룹니다. 지금의 언어 모델이 단어를 다루는 방식의 바탕입니다.

왜 배우나: 오늘의 학습 경로



개념마다 정의와 식을 먼저 세우고, "The fat cat sat on the mat"이라는 한 문장으로 Word2Vec의 계산을 끝까지 따라갑니다. 그다음 위키백과 글과 영화 리뷰로 실제로 학습시켜 결과를 확인합니다. 임베딩을 신경망 안에서 쓰는 방법은 다음 자료(12장 뒷부분)에서 다룹니다.

12-01 NLP에서의 원-핫 인코딩

핵심 한 줄 **원-핫 인코딩은 단어 집합 크기의 벡터에서 그 단어의 번호 자리만 1로 두는 표현이고, 벡터가 너무 길고 단어 사이의 유사도를 나타내지 못한다는 한계가 있다.**

왜 배우나 원-핫 인코딩은 단어를 벡터로 만드는 가장 기본적인 방법이고, 05장과 07장에서 이미 썼습니다. 여기서 다시 보는 이유는 한계를 분명히 하기 위해서입니다. 이 한계가 12장 전체의 출발점입니다.

1) 원-핫 인코딩의 정의

정의

- 단어 집합(vocabulary): 서로 다른 단어들의 집합. book과 books처럼 형태가 조금만 달라도 다른 단어로 봅니다. 크기를 V라고 합니다.
- 원-핫 인코딩(one-hot encoding): 단어 집합의 크기를 벡터의 차원으로 하고, 표현하려는 단어의 번호 자리에 1, 나머지 자리에 0을 넣는 표현 방법. 이렇게 만든 벡터가 원-핫 벡터입니다.

만드는 과정은 두 단계입니다.

1. 단어마다 고유한 번호를 붙입니다(정수 인코딩, 09-05).
2. 길이 V인 벡터에서 그 번호의 자리만 1로, 나머지는 0으로 둡니다.

예시 적용 "나는 자연어 처리를 배운다"를 형태소로 자르면 [나, 는, 자연어, 처리, 를, 배운다] 6개이고, 번호는 나 = 0, 는 = 1, 자연어 = 2, 처리 = 3, 를 = 4, 배운다 = 5입니다(실습 1). "자연어"의 원-핫 벡터는 2번 자리만 1인 [0, 0, 1, 0, 0, 0]입니다.

2) 원-핫 인코딩의 한계

정의

- **한계 1. 벡터가 단어 수만큼 길어진다.** 단어가 1만 개면 모든 단어가 1만 칸짜리 벡터가 되고, 그중 9,999칸은 0입니다. 저장 공간이 낭비됩니다.
- **한계 2. 단어 사이의 유사도를 나타내지 못한다.** 어떤 두 단어의 원-핫 벡터도 1의 자리가 겹치지 않으므로 내적이 항상 0입니다. "강아지"가 "개"와 가까운지 "냉장고"와 가까운지 구분할 수 없습니다.

유사도를 모르면 무엇이 문제일까? 검색을 생각해 봅시다. "삿포로 숙소"를 검색한 사람에게는 "삿포로 게스트 하우스", "삿포로 료칸", "삿포로 호텔"도 보여 줘야 합니다. 숙소, 게스트 하우스, 료칸, 호텔이 비슷한 말이라는 것을 모르면 글자가 똑같은 결과만 찾게 됩니다. 11장의 영화 추천이 틀린 것도 같은 이유였습니다.

예시 적용 "자연어"와 "처리"의 원-핫 벡터 [0, 0, 1, 0, 0, 0]과 [0, 0, 0, 1, 0, 0]의 내적은 0입니다. 어떤 두 단어를 골라도 마찬가지입니다.

단어 집합의 크기	원-핫 벡터의 길이	그중 0인 칸
6 (위의 한 문장)	6	5
10,000	10,000	9,999
500,000	500,000	499,999

비유

교실의 출석 번호표

반 학생 30명을 나타내려고 30칸짜리 표를 만들고, 그 학생의 번호 칸에만 동그라미를 칩니다. 누가 누구인지는 구분되지만, 표만 봐서는 누가 누구와 친한지 알 수 없습니다. 학생이 3만 명인 학교라면 한 명을 나타내는 데 3만 칸이 필요합니다.

번호 칸의 동그라미 = = 원-핫 벡터의 1
표의 칸 수 = 학생 수 = = 벡터의 길이 = 단어 집합의 크기
친한 사이를 알 수 없음 = = 유사도를 나타내지 못함

실습 1

원-핫 벡터 만들기와 두 가지 한계

```
# 코랩에서는 먼저:  !pip install konlpy
from konlpy.tag import Okt

okt = Okt()
tokens = okt.morphs("나는 자연어 처리를 배운다")
print("토큰:", tokens)

# (1) 정수 인코딩: 단어마다 고유한 번호를 붙인다
word2index = {}
for word in tokens:
    if word not in word2index:
        word2index[word] = len(word2index)
print("단어 집합:", word2index)

# (2) 그 번호의 자리만 1, 나머지는 0인 벡터를 만든다
def one_hot_encoding(word, word2index):
    one_hot_vector = [0] * len(word2index)
    one_hot_vector[word2index[word]] = 1
    return one_hot_vector

for word in ["자연어", "처리", "배운다"]:
    print(f"{word:4s} → {one_hot_encoding(word, word2index)}")

# 한계 1: 벡터의 길이 = 단어 집합의 크기
for vocab_size in [6, 10000, 500000]:
    zeros = vocab_size - 1
    print(f"단어 {vocab_size:>7,}개 → 벡터 길이 {vocab_size:>7,},",
          f"그중 0은 {zeros:,}칸")

# 한계 2: 어떤 두 단어의 원-핫 벡터도 겹치는 칸이 없다 (내적 0)
a, b = one_hot_encoding("자연어", word2index), one_hot_encoding("처리", word2index)
print("'자연어'·'처리' 내적:", sum(x * y for x, y in zip(a, b)))
```

실행 결과

```
토큰: ['나', '는', '자연어', '처리', '를', '배운다']
단어 집합: {'나': 0, '는': 1, '자연어': 2, '처리': 3, '를': 4, '배운다': 5}
자연어  → [0, 0, 1, 0, 0, 0]
처리   → [0, 0, 0, 1, 0, 0]
배운다  → [0, 0, 0, 0, 0, 1]
단어       6개 → 벡터 길이       6, 그중 0은 5칸
단어  10,000개 → 벡터 길이  10,000, 그중 0은 9,999칸
단어 500,000개 → 벡터 길이 500,000, 그중 0은 499,999칸
'자연어'·'처리' 내적: 0
```

직접 해 보기 문장을 "나는 자연어 처리를 배운다 나는 파이토치를 배운다"로 바꾸면 단어 집합의 크기와 원-핫 벡터의 길이가 얼마가 되나요?

12-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 원-핫 벡터의 길이는 단어 집합의 크기와 같다. (O / X) 이름 _____

2 원-핫 벡터로 두 단어의 뜻이 비슷한지 알 수 있다. (O / X) 이름 _____

3 원-핫 인코딩의 첫 단계는? 이름 _____

① 단어마다 고유한 번호 붙이기 ② 패딩 ③ 불용어 제거

4 단어가 1만 개일 때 원-핫 벡터 하나에서 0인 칸은 몇 개인가요? 이름 _____

① 1개 ② 9,999개 ③ 10,000개

5 서로 다른 두 단어의 원-핫 벡터의 내적은 항상 _____이다. 이름 _____

6 실습에서 "자연어"의 원-핫 벡터에서 1이 있는 자리는 _____번이다. 이름 _____

7 원-핫 인코딩을 정의하고, "삿포로 숙소" 검색 예시로 한계를 설명해 보세요. 이름 _____

원문: 12-01 NLP에서의 원-핫 인코딩 — https://wikidocs.net/60853

12-02 워드 임베딩

핵심 한 줄

워드 임베딩은 단어를 길이가 짧고 값이 실수인 밀집 벡터로 나타내는 방법이고, 그 값은 사람이 정하지 않고 데이터에서 학습한다.

왜 배우나

원-핫 벡터의 두 한계(너무 길다, 유사도를 모른다)를 한 번에 푸는 방법이 워드 임베딩입니다. 12장의 나머지는 모두 "이 벡터를 어떻게 학습할 것인가"에 대한 이야기이므로, 먼저 무엇을 만들려는 것인지 분명히 합니다.

1) 희소 표현과 밀집 표현

정의

- 희소 표현(sparse representation): 벡터나 행렬의 값 대부분이 0인 표현. 원-핫 벡터와 11장의 DTM이 희소 표현입니다.
- 밀집 표현(dense representation): 벡터의 차원을 단어 집합의 크기가 아니라 **사용자가 정한 값**으로 맞추고, 모든 칸이 **실수 값**을 가지는 표현. 이런 벡터를 밀집 벡터라고 합니다.
- 워드 임베딩(word embedding): 단어를 밀집 벡터로 나타내는 방법. 그렇게 얻은 벡터를 임베딩 벡터라고 합니다. 방법으로는 LSA, Word2Vec, FastText, GloVe가 있고, 파이토치의 `nn.Embedding` 처럼 신경망을 학습하면서 함께 배우는 방법도 있습니다(다음 자료).

	원-핫 벡터	임베딩 벡터
차원	고차원 (단어 집합의 크기)	저차원 (보통 50~300으로 사용자가 정함)
다른 이름	희소 벡터의 일종	밀집 벡터의 일종
값을 정하는 방법	규칙대로 (번호 자리에 1)	훈련 데이터로부터 학습
값의 종류	1과 0	실수
단어 사이의 유사도	항상 0	뜻이 비슷하면 큼

어떻게 짧은 벡터에 더 많은 것을 담을까? 원-핫 벡터는 한 칸이 단어 하나를 맡으므로 단어 수만큼 칸이 필요합니다. 밀집 벡터는 한 칸이 단어 하나를 맡지 않습니다. 칸마다 실수 값을 가지므로, 50칸이면 값의 조합으로 수많은 단어를 서로 다르게 나타낼 수 있습니다. 그리고 비슷한 단어는 비슷한 조합을 갖게 됩니다(11-01의 분산 표현).

예시 적용

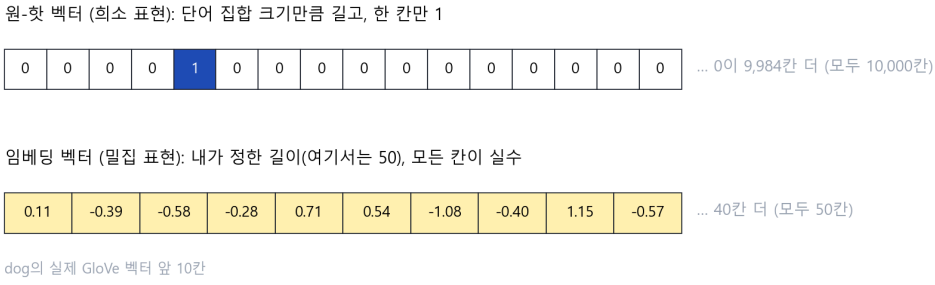


그림 1 위: 단어가 1만 개일 때의 원-핫 벡터. 아래: dog의 실제 임베딩 벡터(GloVe 50차원)의 앞 10칸

dog, cat, computer, netbook, book 다섯 단어의 유사도를 두 표현으로 재 봤습니다(실습 2). 임베딩 벡터는 위키백과 등 60억 단어로 미리 학습해 둔 GloVe 50차원 벡터입니다.

단어 쌍	원-핫 벡터의 유사도	임베딩 벡터의 유사도
dog - cat	0.00	0.92
cat - computer	0.00	0.35
computer - netbook	0.00	0.39
netbook - book	0.00	0.04

원-핫 벡터는 모두 0이지만, 임베딩 벡터는 dog와 cat이 가장 가깝고 netbook과 book은 글자가 비슷해도 멀다고 답합니다. 이 GloVe 임베딩은 단어 40만 개를 담고 있습니다. 원-핫이었다면 단어 하나에 40만 칸이 필요했을 것을 50칸으로 나타냅니다.

비유

색을 나타내는 두 가지 방법

색 1,000가지에 1번부터 1,000번까지 번호를 붙일 수도 있고, 빨강·초록·파랑의 세기 세 숫자(RGB)로 나타낼 수도 있습니다. 번호로는 37번과 38번이 비슷한 색인지 알 수 없습니다. RGB로는 세 숫자만으로 수많은 색을 나타내고, 숫자가 비슷하면 색도 비슷합니다.

색 번호 = = 원-핫 벡터 (국소 표현)
RGB 세 숫자 = = 임베딩 벡터 (밀집 표현)
숫자가 비슷하면 색도 비슷 = = 벡터가 비슷하면 뜻도 비슷

실습 2

원-핫 벡터와 임베딩 벡터의 유사도 비교

```python
import gensim.downloader as api
import torch

words = ["dog", "cat", "computer", "netbook", "book"]
pairs = [("dog", "cat"), ("cat", "computer"),
         ("computer", "netbook"), ("netbook", "book")]

# 1) 희소 표현: 원-핫 벡터 (길이 = 단어 수, 값은 0과 1)
onehot = {w: torch.eye(len(words))[i] for i, w in enumerate(words)}
print("원-핫 dog:", onehot["dog"].tolist())
for a, b in pairs:
    sim = torch.cosine_similarity(onehot[a], onehot[b], dim=0)
    print(f"  원-핫 유사도 {a}-{b}: {sim.item():.2f}")

# 2) 밀집 표현: 학습된 임베딩 벡터 (길이는 내가 정한 차원, 값은 실수)
#    위키백과 등 60억 단어로 미리 학습해 둔 GloVe 50차원 벡터
#    (처음 한 번 66MB 다운로드)
glove = api.load("glove-wiki-gigaword-50")
print("\n임베딩 행렬:", glove.vectors.shape, "= (단어 수, 차원)")
print("임베딩 dog 앞 6칸:", [f"{v:.2f}" for v in glove["dog"][:6]], "… 모두 50칸")
for a, b in pairs:
    print(f"  임베딩 유사도 {a}-{b}: {glove.similarity(a, b):.2f}")

# 3) 크기 비교: 단어 40만 개를 원-핫으로 나타내면 한 단어에 40만 칸
V, D = glove.vectors.shape
print(f"\n한 단어를 나타내는 칸 수: 원-핫 {V:,}칸 / 임베딩 {D}칸")
```

```
실행 결과
원-핫 dog: [1.0, 0.0, 0.0, 0.0, 0.0]
  원-핫 유사도 dog-cat: 0.00
  원-핫 유사도 cat-computer: 0.00
  원-핫 유사도 computer-netbook: 0.00
  원-핫 유사도 netbook-book: 0.00

임베딩 행렬: (400000, 50) = (단어 수, 차원)
임베딩 dog 앞 6칸: ['0.11', '-0.39', '-0.58', '-0.28', '0.71', '0.54'] … 모두 50칸
  임베딩 유사도 dog-cat: 0.92
```

```
   임베딩 유사도 cat-computer: 0.35
   임베딩 유사도 computer-netbook: 0.39
   임베딩 유사도 netbook-book: 0.04

한 단어를 나타내는 칸 수: 원-핫 400,000칸 / 임베딩 50칸
```

직접 해 보기 `pairs`에 ("ship", "boat"), ("ship", "banana")를 추가해 보세요. 11장에서 글자가 달라 전혀 다른 단어였던 ship과 boat의 유사도는 얼마인가요?

확인 문제

12-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 원-핫 벡터는 희소 벡터의 일종이다. (O / X) 이름 _____

2 임베딩 벡터의 길이는 단어 집합의 크기와 같아야 한다. (O / X) 이름 _____

3 임베딩 벡터의 값은 어떻게 정해지나요? 이름 _____
① 사람이 하나씩 정한다 ② 훈련 데이터로부터 학습한다 ③ 항상 0과 1이다

4 단어를 밀집 벡터로 나타내는 방법을 무엇이라고 하나요? 이름 _____
① 워드 임베딩 ② 원-핫 인코딩 ③ 패딩

5 값 대부분이 0인 표현을 _____ 표현이라고 한다. 이름 _____

6 실습에서 임베딩 벡터로 잰 dog와 cat의 유사도는 _____였다. 이름 _____

7 희소 표현과 밀집 표현을 정의하고, 색 번호와 RGB에 빗대어 차이를 설명해 보세요. 이름 _____

원문: 12-02 워드 임베딩 — https://wikidocs.net/60852 (임베딩 벡터의 유사도 비교는 이 자료에서 추가했습니다)

12-03 워드투벡터(Word2Vec)

핵심 한 줄

Word2Vec은 "비슷한 자리에 나오는 단어는 뜻이 비슷하다"는 가정 아래, 주변 단어로 중심 단어를 맞히는(또는 그 반대의) 문제를 풀게 하면서 단어의 임베딩 벡터를 학습하는 방법이다.

왜 배우나

임베딩 벡터가 좋다는 것은 알았습니다. 그런데 "강아지와 고양이는 비슷하다"는 정답표가 없는데 어떻게 학습할까요? Word2Vec의 답은 글 자체를 정답으로 쓰는 것입니다. 문장에서 단어 하나를 가리고 주변 단어로 맞히게 하면, 따로 정답을 만들지 않고도 학습할 수 있습니다. 이 생각은 뒤에 나올 BERT, GPT의 학습 방법으로 그대로 이어집니다.

1) 분포 가설과 분산 표현

정의

- 분포 가설(distributional hypothesis): **비슷한 위치(문맥)에 등장하는 단어들은 비슷한 뜻을 가진다**는 가정
- 분산 표현(distributed representation): 분포 가설을 이용해 단어의 뜻을 벡터의 여러 차원에 나눠 담은 표현. 저차원이고 실수 값이므로 12-02의 임베딩 벡터입니다.
- Word2Vec: 분산 표현을 학습하는 대표적인 방법. 학습 방식은 **CBOW**와 **Skip-gram** 두 가지입니다.

왜 주변 단어를 보면 뜻을 알 수 있을까? "강아지" 근처에는 "귀엽다", "산책", "짖다"가 자주 나옵니다. "고양이" 근처에도 "귀엽다"가 자주 나옵니다. 주변에 오는 단어가 비슷하다면 두 단어는 비슷한 역할을 하는 말입니다. 주변 단어를 잘 맞히도록 벡터를 고쳐 가면, 비슷한 주변을 가진 단어들은 비슷한 벡터를 갖게 됩니다.

예시 적용

잘 학습된 Word2Vec 벡터는 덧셈과 뺄셈이 뜻의 계산처럼 동작합니다. 교재가 소개하는 한국어 예로 "한국 - 서울 + 도쿄 = 일본"이 있습니다. 이 자료에서 직접 학습한 영어 모델에서도 "king - man + woman"에 가장 가까운 단어가 queen으로 나왔습니다(12-04 실습 5).

2) 학습 데이터: 중심 단어, 주변 단어, 윈도우

정의

- 중심 단어(center word): 지금 보고 있는 단어. 주변 단어(context word): 중심 단어의 앞뒤에 있는 단어
- 윈도우(window): 중심 단어의 앞뒤로 몇 개의 단어를 볼지 정한 범위. 윈도우 크기가 n이면 주변 단어는 최대 $2n$개입니다.
- 슬라이딩 윈도우: 윈도우를 한 단어씩 옮기며 중심 단어를 바꿔 가면서 학습 데이터를 만드는 방법(07-03 심화에서 10글자 창을 옮긴 것과 같은 방법)
- **CBOW**(Continuous Bag of Words)는 주변 단어들로 중심 단어를 맞히고, **Skip-gram**은 중심 단어로 주변 단어들을 맞힙니다.

예시 적용

문장 "The fat cat sat on the mat"에서 윈도우 크기 2로 학습 데이터를 만듭니다(실습 3).



그림 2 윈도우를 한 칸씩 옮기면 중심 단어(노랑)와 주변 단어(파랑)가 바뀐다. 문장의 양 끝에서는 주변 단어가 2~3개뿐이다

중심 단어가 sat일 때 주변 단어는 fat, cat, on, the입니다. CBOW는 "[fat, cat, on, the] → sat" 한 문제로 쓰고, Skip-gram은 "sat → fat", "sat → cat", "sat → on", "sat → the" 네 문제로 씁니다. 이 한 문장에서 CBOW의 학습 샘플은 7개, Skip-gram은 22개가 나옵니다.

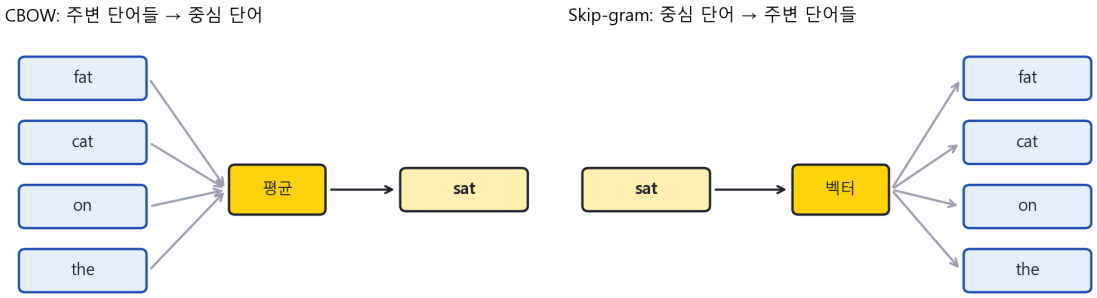


그림 3 CBOW와 Skip-gram은 입력과 정답이 서로 반대다

3) CBOW의 계산 과정

정의

단어 집합의 크기를 V, 임베딩 차원(투사층의 크기)을 M, 윈도우 크기를 n이라고 합니다. CBOW는 가중치 행렬 두 개를 학습합니다.

- W: 입력층과 투사층 사이의 가중치. 모양 $(V,\ M)$
- W': 투사층과 출력층 사이의 가중치. 모양 $(M,\ V)$. W를 뒤집은 것이 아니라 **별개의 행렬**입니다.

$$\underbrace{x_c}_{\text{one-hot }(V)} \xrightarrow{\times W} \underbrace{W_c}_{(M)} \xrightarrow{\text{mean}} \underbrace{v}_{(M)} \xrightarrow{\times W'} \underbrace{z}_{\text{score }(V)} \xrightarrow{\text{softmax}} \underbrace{\hat{y}}_{(V)}$$

1. **룩업**: 주변 단어 c의 원-핫 벡터 x_c에 W를 곱합니다. 원-핫 벡터는 한 자리만 1이므로, 곱한 결과는 W의 그 번호 행을 그대로 꺼낸 것과 같습니다. 그래서 이 연산을 룩업(lookup), W를 룩업 테이블이라고 합니다.
2. **평균**: 주변 단어 $2n$개에서 꺼낸 벡터들의 평균 v를 구합니다. 이 층을 투사층(projection layer)이라고 합니다. 활성화 함수가 없습니다.
3. **점수**: v에 W'를 곱해 단어 집합의 모든 단어에 대한 점수 z를 얻습니다.
4. **확률**: 소프트맥스로 합이 1인 확률 $\hat{y}$로 바꿉니다. $\hat{y}_j$는 j번 단어가 중심 단어일 확률입니다.
5. **손실**: 정답인 중심 단어의 원-핫 벡터 y와 비교해 크로스 엔트로피를 구합니다(05장).

$$\text{loss} = -\sum_{j=1}^{V} y_j \log \hat{y}_j = -\log \hat{y}_{c^*}$$

y는 정답 단어 c^*의 자리만 1이므로, 합에서 그 항 하나만 남습니다. 정답 단어에 준 확률이 1에 가까울수록 손실은 0에 가까워집니다. 이 손실을 줄이도록 역전파로 W와 W'를 고칩니다.

임베딩 벡터는 어디에 있을까? 학습이 끝난 W의 i번째 행이 i번 단어의 임베딩 벡터입니다. 다음 단어를 맞히는 것 자체가 목적이 아니라, 맞히도록 학습하는 과정에서 만들어진 W가 얻고자 하는 결과물입니다.

Word2Vec은 딥 러닝 모델이 아닙니다. 은닉층이 투사층 하나뿐이고 활성화 함수도 없는 얕은 신경망입니다. 계산이 단순해서 큰 말뭉치를 빠르게 학습할 수 있습니다.

Skip-gram은 무엇이 다를까? 입력이 중심 단어 하나이므로 평균을 내는 단계가 없습니다. 중심 단어의 벡터로 주변 단어 각각을 맞힙니다. 나머지는 같습니다. 여러 실험에서 Skip-gram이 CBOW보다 전반적으로 성능이 좋다고 알려져 있습니다.

예시 적용

예문의 서로 다른 단어는 7개(The와 the를 다른 단어로 봄)이므로 $V = 7$, 임베딩 차원은 $M = 5$로 정합니다. W는 (7, 5), W'는 (5, 7)입니다. 주변 단어 [fat, cat, on, the]로 sat을 맞히는 계산을 따라갑니다(실습 4).

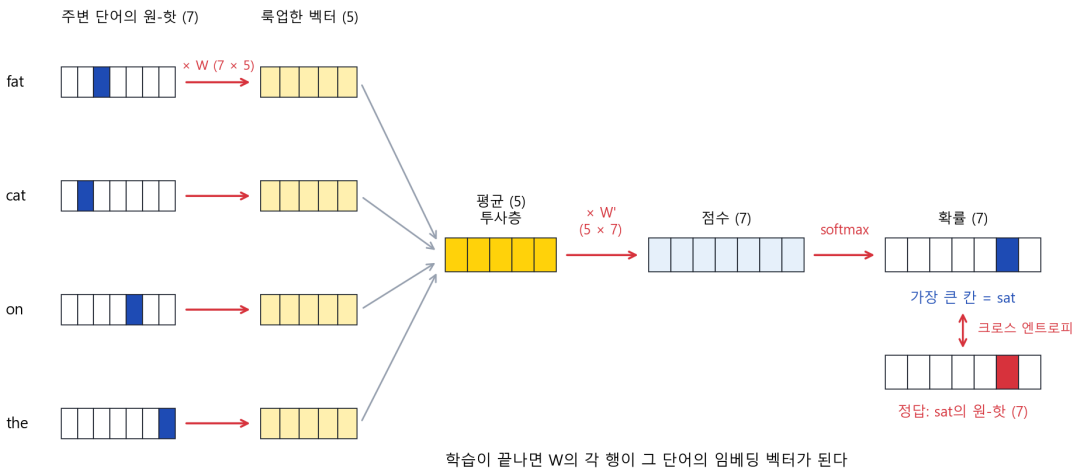


그림 4 주변 단어 4개의 원-핫 벡터(7칸)가 W에서 5칸짜리 벡터를 하나씩 꺼내고, 평균을 낸 뒤 W'를 곱해 7칸짜리 점수가 된다. 소프트맥스 후 정답 sat의 원-핫 벡터와 비교한다

단계	모양	실습에서 확인한 것
룩업	(4, 5): 주변 단어 4개, 각 5칸	원-핫 × W = W의 그 행 (True)
평균	(5,)	
점수, 확률	(7,)	학습 전 $P(\text{sat}) = 0.157$, 손실 1.852
50번 학습한 뒤		$P(\text{sat}) = 0.992$, 예측 sat

학습 전에는 7개 중 찍는 수준($1/7 = 0.143$)과 비슷했고, 학습 뒤에는 sat을 맞힙니다. 이때 W의 cat 행 [0.64, -0.81, -0.70, 1.02, -1.38]이 cat의 임베딩 벡터입니다. 실제로는 문장 하나가 아니라 수백만 문장의 모든 윈도우에 대해 이 과정을 반복합니다.

4) 네거티브 샘플링

정의

문제: 느리다. 위의 방법은 한 번 학습할 때마다 단어 집합의 **모든 단어**에 대해 점수를 구하고 소프트맥스를 계산한 뒤, 모든 단어의 벡터를 고칩니다. V가 수십만이면 "강아지"를 학습하는 데 아무 상관없는 "돈가스", "컴퓨터"의 벡터까지 매번 계산하게 됩니다.

네거티브 샘플링(negative sampling)은 이 문제를 풀기 위해 V개 중 하나를 고르는 문제를 **O/X 문제 몇 개**로 바꿉니다.

- 실제로 주변에 나온 단어 쌍(강아지, 고양이)에는 정답 1을 줍니다.
- 무작위로 뽑은, 주변에 나오지 않은 단어 쌍(강아지, 돈가스)에는 정답 0을 줍니다. 이것이 네거티브 샘플입니다.
- 모델은 두 단어 벡터의 내적에 시그모이드를 씌워 "주변 단어가 맞을 확률"을 내고(04장 로지스틱 회귀), 그 쌍에 나온 단어의 벡터만 고칩니다.

한 번에 고치는 단어가 V개에서 "정답 1개 + 네거티브 샘플 몇 개"로 줄어듭니다. 실제로 Word2Vec을 쓴다고 하면 대개 Skip-gram에 네거티브 샘플링을 더한 SGNS(Skip-Gram with Negative Sampling)를 뜻합니다.

예시 적용



그림 5 왼쪽은 단어 집합 전체를 매번 고친다. 오른쪽은 실제 주변 단어와 무작위로 뽑은 몇 개만 고친다

12-04에서 학습할 모델은 단어가 71,290개입니다. 소프트맥스라면 윈도우 하나를 학습할 때마다 71,290개 단어의 점수를 모두 계산해야 합니다. 네거티브 샘플을 5개 쓰면(gensim의 기본값) 6개만 계산하면 됩니다.

비유

빈칸 채우기 문제집

영어 단어의 뜻을 사전 없이 익히는 방법이 있습니다. 문장에서 단어 하나를 가리고 앞뒤를 보고 맞히는 문제를 수없이 푸는 것입니다. "I drink ___ every morning"의 빈칸에 coffee도 tea도 들어갈 수 있다는 것을 여러 번 겪으면, 두 단어가 비슷한 종류의 말이라는 감이 생깁니다.

가린 단어	= = 중심 단어
앞뒤 단어	= = 주변 단어 (윈도우)
문제를 풀며 생긴 감	= = 임베딩 벡터 (W의 행)
같은 빈칸에 들어가는 단어들	= = 비슷한 벡터를 갖는 단어들

실습 3

슬라이딩 윈도우로 학습 데이터 만들기

```
sentence = "The fat cat sat on the mat".split()
window = 2                      # 중심 단어의 앞뒤로 몇 개씩 볼지

# 윈도우를 한 칸씩 옮기며(슬라이딩 윈도우) 학습 데이터를 만든다
cbow, skipgram = [], []
for i, center in enumerate(sentence):
    left = sentence[max(0, i - window):i]
    right = sentence[i + 1:i + 1 + window]
    context = left + right
    cbow.append((context, center))                    # 주변 단어들 → 중심 단어
    skipgram += [(center, c) for c in context]        # 중심 단어 → 주변 단어 하나씩

print("CBOW: 주변 단어들로 중심 단어 맞히기")
for context, center in cbow:
    print(f"  {str(context):32s} → {center}")

print(f"\nSkip-gram: 중심 단어로 주변 단어 맞히기 (모두 {len(skipgram)}쌍, 앞 6쌍)")
for center, c in skipgram[:6]:
    print(f"  {center:4s} → {c}")
print(f"\n학습 샘플 수: CBOW {len(cbow)}개, Skip-gram {len(skipgram)}개")
```

실행 결과

```
CBOW: 주변 단어들로 중심 단어 맞히기
  ['fat', 'cat']                  → The
  ['The', 'cat', 'sat']           → fat
  ['The', 'fat', 'sat', 'on']     → cat
  ['fat', 'cat', 'on', 'the']     → sat
  ['cat', 'sat', 'the', 'mat']    → on
  ['sat', 'on', 'mat']            → the
  ['on', 'the']                   → mat

Skip-gram: 중심 단어로 주변 단어 맞히기 (모두 22쌍, 앞 6쌍)
  The  → fat
  The  → cat
  fat  → The
  fat  → cat
  fat  → sat
  cat  → The

학습 샘플 수: CBOW 7개, Skip-gram 22개
```

직접 해 보기 `window`를 1로 바꾸면 Skip-gram의 학습 쌍은 몇 개가 되나요? 윈도우가 작으면 무엇을 덜 배우게 될까요?

실습 4 CBOW 한 걸음을 식 그대로 계산하기

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
sentence = "The fat cat sat on the mat".split()
# ['The', 'cat', 'fat', 'mat', 'on', 'sat', 'the']
vocab = sorted(set(sentence))
w2i = {w: i for i, w in enumerate(vocab)}
V, M = len(vocab), 5                            # 단어 집합 크기 7, 임베딩 차원 5

# 입력 쪽 가중치 (V, M): 이것이 임베딩 행렬
W = nn.Parameter(torch.randn(V, M) * 0.5)
W2 = nn.Parameter(torch.randn(M, V) * 0.5)      # 출력 쪽 가중치 (M, V)

# 1) 원-핫 벡터 × W 는 W의 한 행을 꺼내는 것과 같다 (룩업)
x = torch.eye(V)[w2i["cat"]]                    # cat의 원-핫 (7,)
print("원-핫 × W = W의 cat 행?", torch.allclose(x @ W, W[w2i["cat"]]))

# 2) 순전파: 주변 단어 4개로 가운데 'sat' 맞히기
context = ["fat", "cat", "on", "the"]
idx = torch.tensor([w2i[w] for w in context])

def forward(idx):
    looked = W[idx]                             # (4, 5): 주변 단어마다 벡터 하나씩
    v = looked.mean(dim=0)                      # (5,): 평균 (투사층)
    z = v @ W2                                  # (7,): 단어마다 점수
    return looked, v, z

looked, v, z = forward(idx)
print("룩업", tuple(looked.shape), "→ 평균", tuple(v.shape),
      "→ 점수", tuple(z.shape))
prob = torch.softmax(z, dim=0)
p_sat = prob[w2i["sat"]]
print(f"학습 전 P(sat) = {p_sat.item():.3f}, 손실 = {-p_sat.log().item():.3f}")

# 3) 학습: 손실(크로스 엔트로피)을 줄이도록 W와 W2를 고친다
optimizer = torch.optim.SGD([W, W2], lr=0.5)
target = torch.tensor(w2i["sat"])
for step in range(50):
    _, _, z = forward(idx)
    loss = nn.functional.cross_entropy(z, target)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
prob = torch.softmax(forward(idx)[2], dim=0)
p_sat = prob[w2i["sat"]].item()
print(f"학습 후 P(sat) = {p_sat:.3f}, 예측: {vocab[prob.argmax()]}")
cat_vec = [f"{v:.2f}" for v in W[w2i["cat"]].tolist()]
print("cat의 임베딩 벡터 (W의 cat 행):", cat_vec)
```

실행 결과

```
원-핫 × W = W의 cat 행? True
룩업 (4, 5) → 평균 (5,) → 점수 (7,)
학습 전 P(sat) = 0.157, 손실 = 1.852
학습 후 P(sat) = 0.992, 예측: sat
cat의 임베딩 벡터 (W의 cat 행): ['0.64', '-0.81', '-0.70', '1.02', '-1.38']
```

직접 해 보기 `M`을 2로 줄이면 룩업, 평균의 모양이 어떻게 바뀌나요? 점수의 모양은 왜 그대로 (7,)일까요?

확인 문제

12-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 CBOW는 주변 단어들로 중심 단어를 맞힌다. (O / X) 이름 _____

2 Word2Vec은 은닉층을 여러 개 쌓은 깊은 신경망이다. (O / X) 이름 _____

3 학습이 끝난 뒤 단어의 임베딩 벡터로 쓰는 것은? 이름 _____

① 소프트맥스의 출력 ② 가중치 행렬 W의 각 행 ③ 정답 원-핫 벡터

4 네거티브 샘플링을 쓰는 이유는? 이름 _____

① 단어 집합 전체를 매번 계산하면 너무 느려서 ② 단어 집합을 늘리려고 ③ 문장을 짧게 하려고

5 "비슷한 위치에 등장하는 단어는 비슷한 뜻을 가진다"는 가정을 _____ 가설이라고 한다. 이름 _____

6 윈도우 크기가 2일 때 중심 단어 하나의 주변 단어는 최대 _____개이다. 이름 _____

7 CBOW에서 원-핫 벡터가 확률이 되기까지의 단계(룩업, 평균, 점수, 소프트맥스)를 벡터의 모양과 함께 설명해 보세요. 이름 _____

원문: 12-03 워드투벡터(Word2Vec) — https://wikidocs.net/60854

12-04 Word2Vec 학습 및 시각화

핵심 한 줄

gensim의 Word2Vec에 토큰화한 문장들을 넣으면 임베딩 행렬 (단어 수, 차원)이 만들어지고, 그 벡터로 비슷한 단어를 찾거나 2차원에 그려 볼 수 있다.

왜 배우나

12-03의 계산을 직접 짤 필요는 없습니다. 이미 빠르게 구현된 도구가 있습니다. 여기서는 그 도구에 무엇을 넣고, 어떤 설정값을 주고, 결과를 어떻게 확인하는지 익힙니다. 데이터의 양과 종류에 따라 결과가 얼마나 달라지는지도 함께 봅니다.

1) gensim으로 Word2Vec 학습하기

정의

글 $\xrightarrow{\text{전처리, 토큰화 (09장)}}$ 토큰 리스트의 리스트 $\xrightarrow{\texttt{Word2Vec}}$ 임베딩 행렬 $(V,\ M)$

gensim은 Word2Vec, FastText 등을 제공하는 파이썬 패키지입니다. 입력은 **문장마다 토큰으로 잘라 둔 리스트**이고, 중요한 설정값은 다섯 가지입니다.

설정값	뜻	정하는 기준
`vector_size`	임베딩 벡터의 차원 M	보통 100~300. 크면 더 많이 담지만 데이터가 많이 필요함
`window`	윈도우 크기 (앞뒤로 볼 단어 수)	작으면 문법적으로 비슷한 단어, 크면 주제가 비슷한 단어가 모임
`min_count`	이 횟수보다 적게 나온 단어는 학습하지 않음	드문 단어는 주변 단어의 예가 적어 벡터를 제대로 배울 수 없음 (09-02)
`workers`	학습에 쓸 CPU 코어 수	많을수록 빠름
`sg`	0이면 CBOW, 1이면 Skip-gram	Skip-gram이 대체로 좋지만 느림

- `model.wv.most_similar("단어")` : 코사인 유사도(11-04)가 큰 단어들을 돌려줍니다.
- `model.wv.most_similar(positive=[...], negative=[...])` : 벡터를 더하고 뺀 결과와 가장 가까운 단어를 돌려줍니다.
- `model.wv.save_word2vec_format("파일")` , `KeyedVectors.load_word2vec_format("파일")` : 학습한 벡터를 저장하고 다시 불러옵니다.

예시 적용

영어. 교재는 TED 강연 자막 약 27만 문장으로 학습합니다. 이 자료는 내려받기 쉬운 위키백과 발췌본 text8(1,700만 단어)로 같은 설정(100차원, 윈도우 5, CBOW)을 썼습니다(실습 5). 임베딩 행렬은 (71290, 100)입니다. 5번 이상 나온 단어 71,290개가 100칸짜리 벡터를 하나씩 가집니다.

질문	결과 (유사도)
man과 비슷한 단어	woman (0.72), girl (0.65), creature (0.63), boy (0.57), stranger (0.57)
seoul과 비슷한 단어	pune (0.77), nanjing (0.76), osaka (0.75), nairobi (0.74), prefecture (0.74)
king - man + woman	queen (0.68), empress (0.66), prince (0.62)

"사람을 뜻하는 말", "도시 이름" 같은 정답표를 준 적이 없는데도 비슷한 종류의 단어가 모였습니다. 주변 단어를 맞히는 문제만 풀었을 뿐입니다. 학습은 무작위로 시작하고 여러 코어가 동시에 계산하므로, 실행할 때마다 숫자와 순서가 조금씩 달라집니다.

한국어. 네이버 영화 리뷰를 09장의 방법(한글만 남기기, Okt로 형태소 토큰화, 불용어 제거)으로 전처리한 뒤 학습합니다(실습 6). 교재는 리뷰 20만 개를 모두 쓰고, 이 자료는 시간을 줄이려고 3만 개만 썼습니다. 임베딩 행렬은 (5200, 100)입니다.

단어	비슷한 단어	해석
배우	연기, 연기력, 연출, 스토리, 모두, 탄탄하다	영화를 평가할 때 함께 쓰는 말
재미	스릴, 나름, 젬, 긴장감, 보람, 가볍다	"젬"은 "잼"(재미)의 변형. 비슷한 자리에 쓰여 가깝게 나옴
최고	꼽는, 근래, 단연, 중, 생애, 산악	뜻이 비슷한 말이 아니라 "근래 최고", "단연 최고", "생애 최고"처럼 **옆에 붙어 다니는 말**

잘 안 되는 경우. "최고"의 결과는 비슷한 말이 아니라 함께 쓰이는 말입니다. Word2Vec은 "비슷한 자리에 나오는 단어"를 가깝게 만드는데, 데이터가 적으면 자주 붙어 다니는 단어와 구분하지 못합니다. 교재가 리뷰 20만 개로 학습한 모델에서는 "최민식"과 비슷한 단어로 한석규, 안성기, 김수현 같은 배우 이름이 나옵니다. 임베딩의 품질은 데이터의 양에 크게 좌우됩니다.

2) 임베딩 벡터의 시각화와 사전 훈련된 임베딩

정의

- 차원 축소: 100차원 벡터는 눈으로 볼 수 없으므로, 벡터들 사이의 관계를 될 수 있는 대로 유지하면서 2~3차원으로 줄이는 것. 대표적인 방법이 PCA와 t-SNE입니다.
- 임베딩 프로젝터(embedding projector): 구글이 제공하는 임베딩 시각화 도구(https://projector.tensorflow.org). 학습한 모델을 `python -m gensim.scripts.word2vec2tensor --input 모델 --output 모델` 로 변환하면 벡터 파일(`_tensor.tsv`)과 단어 파일(`_metadata.tsv`)이 생기고, 사이트의 Load 버튼으로 두 파일을 올리면 3차원으로 돌려 볼 수 있습니다.
- 사전 훈련된 워드 임베딩(pre-trained word embedding): 방대한 글로 미리 학습해 공개된 임베딩 벡터. 직접 학습할 데이터가 적을 때 가져다 씁니다(다음 자료 12-07).

차원을 줄인 그림을 볼 때 주의할 점. 100개의 숫자를 2개로 줄였으므로 정보가 많이 사라집니다. 그림에서 가깝다고 실제 벡터가 가까운 것은 아닐 수 있습니다. 정확한 값은 코사인 유사도로 확인하고, 그림은 전체 모양을 훑어보는 데 씁니다.

예시 적용

실습 5에서 학습한 모델에서 네 종류의 단어 24개를 골라 PCA로 2차원에 그렸습니다.

직접 학습한 Word2Vec: 비슷한 종류의 단어가 가까이 모인다 (100차원 → 2차원)

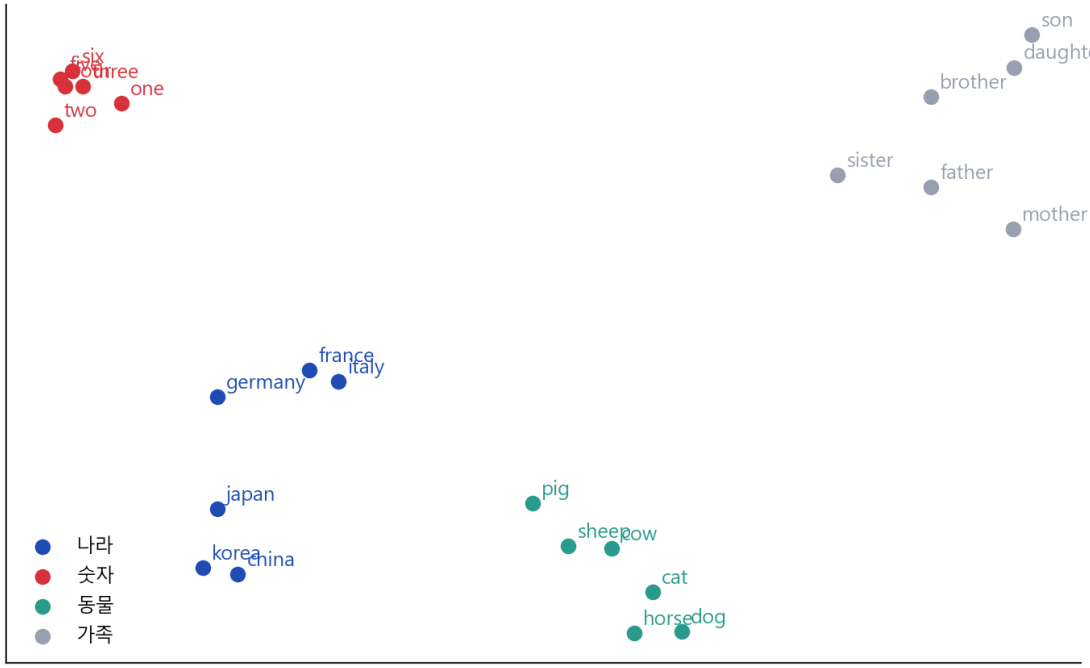


그림 6 나라, 숫자, 동물, 가족을 뜻하는 단어가 종류별로 모인다. 색은 그림을 그릴 때 사람이 칠한 것이고, 모델은 종류를 배운 적이 없다

구글이 공개한 사전 훈련된 Word2Vec은 단어 300만 개, 300차원으로 모양이 (3000000, 300)입니다. 숫자 하나가 4바이트이므로 $3{,}000{,}000 \times 300 \times 4$바이트, 약 3.35GB입니다. 임베딩 행렬의 크기는 "단어 수 × 차원"으로 정해집니다.

비유

지도에 도시 찍기

도시마다 위도와 경도 두 숫자를 붙이면, 숫자만 보고도 어느 도시끼리 가까운지 알 수 있습니다. 워드 임베딩은 단어마다 100개의 숫자를 붙여 "뜻의 지도" 위에 찍는 일입니다. 100차원 지도는 볼 수 없으니, 종이에 옮기려면 2차원으로 눌러야 하고 그 과정에서 거리가 조금씩 왜곡됩니다.

도시의 위도, 경도	= = 단어의 임베딩 벡터
가까운 도시	= = 뜻이 비슷한 단어
지구본을 종이 지도로	= = 차원 축소 (PCA)

실습 5

영어 Word2Vec 학습하기

```python
# 교재는 TED 강연 자막(27만 문장)을 쓴다.
# 여기서는 내려받기 쉬운 위키백과 발췌본 text8을 쓴다.
import gzip

import gensim.downloader as api
from gensim.models import KeyedVectors, Word2Vec

# 1) 데이터: 이미 소문자로 바뀌고 구두점이 지워진 1,700만 단어
#    (처음 한 번 31MB 다운로드)
path = api.load("text8", return_path=True)
words = gzip.open(path, "rt", encoding="utf8").read().split()
# 1000단어씩 한 문장으로
sentences = [words[i:i + 1000] for i in range(0, len(words), 1000)]
print("단어 수:", f"{len(words):,}", "/ 문장 수:", f"{len(sentences):,}")
print("앞 10단어:", sentences[0][:10])

# 2) 학습 (CPU로 1~2분)
model = Word2Vec(sentences=sentences,
                 vector_size=100,     # 임베딩 벡터의 차원
                 window=5,            # 중심 단어의 앞뒤로 볼 단어 수
                 min_count=5,         # 5번 미만으로 나온 단어는 버림
                 workers=4,           # 동시에 쓸 CPU 코어 수
                 sg=0)                # 0: CBOW, 1: Skip-gram
print("임베딩 행렬:", model.wv.vectors.shape, "= (단어 수, 차원)")
```

```python
# 3) 비슷한 단어 찾기 (코사인 유사도가 큰 순서)
for word in ["man", "seoul"]:
    similar = [(w, round(s, 2)) for w, s in model.wv.most_similar(word, topn=5)]
    print(f"{word}과 비슷한 단어:", similar)

# 4) 단어 벡터의 덧셈과 뺄셈: king - man + woman
result = model.wv.most_similar(positive=["king", "woman"], negative=["man"], topn=3)
print("king - man + woman ≈", [(w, round(s, 2)) for w, s in result])

# 5) 저장하고 다시 불러오기
model.wv.save_word2vec_format("eng_w2v")
loaded = KeyedVectors.load_word2vec_format("eng_w2v")
best = loaded.most_similar("man", topn=1)[0][0]
print("불러온 모델에서 man과 가장 비슷한 단어:", best)
```

실행 결과

```
단어 수: 17,005,207 / 문장 수: 17,006
앞 10단어: ['anarchism', 'originated', 'as', 'a', 'term', 'of', 'abuse', 'first', 'used', 'against']
임베딩 행렬: (71290, 100) = (단어 수, 차원)
man과 비슷한 단어: [('woman', 0.72), ('girl', 0.65), ('creature', 0.63), ('boy', 0.57), ('stranger', 0.57)]
seoul과 비슷한 단어: [('pune', 0.77), ('nanjing', 0.76), ('osaka', 0.75), ('nairobi', 0.74), ('prefecture', 0.74)]
king - man + woman ≈ [('queen', 0.68), ('empress', 0.66), ('prince', 0.62)]
불러온 모델에서 man과 가장 비슷한 단어: woman
```

직접 해 보기 `sg=0` 을 `sg=1` (Skip-gram)로 바꾸면 학습 시간과 결과가 어떻게 달라지나요?

실습 6 한국어 Word2Vec 학습하기 (네이버 영화 리뷰)

```python
# 코랩에서는 먼저:  !pip install konlpy
# 교재는 리뷰 20만 개를 모두 쓴다.
# 형태소 분석에 시간이 걸려서 여기서는 앞 3만 개만 쓴다(약 3분).
import re
import urllib.request

from gensim.models import Word2Vec
from konlpy.tag import Okt

# 1) 네이버 영화 리뷰 내려받기 (19MB)
url = "https://raw.githubusercontent.com/e9t/nsmc/master/ratings.txt"
urllib.request.urlretrieve(url, filename="ratings.txt")
lines = open("ratings.txt", encoding="utf8").read().split("\n")[1:]
reviews = [line.split("\t")[1] for line in lines if len(line.split("\t")) == 3]
print("전체 리뷰:", f"{len(reviews):,}개 → 사용:", "30,000개")
reviews = reviews[:30000]

# 2) 전처리: 한글만 남기기 → 형태소 토큰화(기본형) → 불용어 제거 (09장)
stopwords = {"의", "가", "이", "은", "들", "는", "좀", "잘", "걍", "과",
             "도", "를", "으로", "자", "에", "와", "한", "하다"}
okt = Okt()
tokenized = []
for review in reviews:
    review = re.sub(r"[^ㄱ-ㅎㅏ-ㅣ가-힣 ]", "", review)
    tokens = okt.morphs(review, stem=True)
    tokenized.append([w for w in tokens if w not in stopwords])
print("첫 리뷰의 토큰:", tokenized[0])
lengths = [len(t) for t in tokenized]
mean_len = round(sum(lengths) / len(lengths), 1)
print("리뷰 길이: 최대", max(lengths), "/ 평균", mean_len)

# 3) 학습
model = Word2Vec(sentences=tokenized, vector_size=100, window=5,
                 min_count=5, workers=4, sg=0)
print("임베딩 행렬:", model.wv.vectors.shape)

# 4) 비슷한 단어
for word in ["배우", "재미", "최고"]:
    similar = [w for w, _ in model.wv.most_similar(word, topn=6)]
    print(f"'{word}'와 비슷한 단어:", similar)
```

실행 결과

```
전체 리뷰: 200,000개 → 사용: 30,000개
첫 리뷰의 토큰: ['어리다', '때', '보고', '지금', '다시', '보다', '재있다', 'ㅋㅋ']
리뷰 길이: 최대 68 / 평균 10.5
임베딩 행렬: (5200, 100)
```

```
'배우'와 비슷한 단어: ['연기', '연기력', '연출', '스토리', '모두', '탄탄하다']
'재미'와 비슷한 단어: ['스릴', '나름', '젬', '긴장감', '보람', '가볍다']
'최고'와 비슷한 단어: ['꼽는', '근래', '단연', '중', '생애', '산악']
```

직접 해 보기 시간이 된다면 `reviews[:30000]`을 `reviews[:100000]`으로 늘려 보세요(약 10분). "최고"와 비슷한 단어가 어떻게 바뀌나요?

확인 문제

12-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 gensim의 Word2Vec에는 토큰화한 문장들의 리스트를 넣는다. (O / X) 이름 _____

2 Word2Vec을 학습하려면 "비슷한 단어" 정답표를 사람이 만들어 줘야 한다. (O / X) 이름 _____

3 `vector_size=100`이 뜻하는 것은? 이름 _____

① 단어를 100개만 학습한다 ② 임베딩 벡터가 100칸이다 ③ 100번 반복 학습한다

4 `sg=1`은 어떤 방식인가요? 이름 _____

① CBOW ② Skip-gram ③ GloVe

5 실습의 영어 모델에서 king - man + woman에 가장 가까운 단어는 _____이었다. 이름 _____

6 100차원 벡터를 그림으로 보려고 2차원으로 줄이는 것을 _____ 축소라고 한다. 이름 _____

7 실습 6에서 "최고"와 비슷한 단어로 "근래", "단연"이 나온 이유를, Word2Vec이 무엇을 가깝게 만드는지와 연결해 설명해 보세요. 이름 _____

원문: 12-04 Word2Vec 학습 및 시각화 — https://wikidocs.net/217114 (교재의 TED 자막을 text8로, 리뷰 20만 개를 3만 개로 바꿔 실행했습니다. 그래서 교재의 출력과 단어, 숫자가 다릅니다)

12-05 글로브와 패스트텍스트

핵심 한 줄 **GloVe는 말뭉치 전체에서 센 "함께 나온 횟수"를 임베딩에 반영하고, FastText는 단어를 글자 조각으로 쪼개 처음 보는 단어와 오타에도 벡터를 만들어 준다.**

왜 배우나 Word2Vec에는 두 가지 아쉬움이 있습니다. 윈도우 안의 단어만 봐서 말뭉치 전체의 통계를 쓰지 못하고, 학습할 때 보지 못한 단어에는 벡터를 줄 수 없습니다. GloVe는 앞의 것을, FastText는 뒤의 것을 고친 방법입니다. 실제로는 이 셋 가운데 하나를 골라 쓰거나 공개된 벡터를 가져다 씁니다.

1) GloVe: 동시 등장 행렬과 손실 함수

정의

GloVe(Global Vectors for Word Representation)는 2014년 스탠퍼드 대학에서 만든 방법으로, 횟수를 세는 방식과 예측하는 방식을 함께 씁니다.

- **기존 방법의 아쉬움**: 횟수 기반인 LSA는 말뭉치 전체의 통계를 쓰지만 "왕 : 남자 = 여왕 : ?" 같은 유추에 약합니다. 예측 기반인 Word2Vec은 유추에 강하지만 윈도우 안만 봐서 전체 통계를 반영하지 못합니다.
- 동시 등장 행렬(co-occurrence matrix) X: 행과 열이 모두 단어 집합의 단어이고, X_{ik}는 단어 i의 윈도우 안에 단어 k가 나온 횟수입니다. i 옆에 k가 나온 횟수와 k 옆에 i가 나온 횟수가 같으므로 대칭입니다.
- 동시 등장 확률 $P(k \mid i) = X_{ik}/X_i$: 단어 i가 나왔을 때 그 주변에 k가 나올 확률. X_i는 i행의 값을 모두 더한 것입니다.

GloVe의 생각. 중심 단어 벡터 w_i와 주변 단어 벡터 $\tilde{w}_k$의 **내적이 두 단어가 함께 나온 횟수의 로그와 같아지도록** 벡터를 학습합니다.

$$w_i^\top \tilde{w}_k + b_i + \tilde{b}_k = \log X_{ik}$$

$w_i^\top \tilde{w}_k$는 내적, b_i와 $\tilde{b}_k$는 단어마다 하나씩 있는 편향입니다. 함께 자주 나온 두 단어는 내적이 커지고, 내적이 크다는 것은 벡터가 비슷한 방향이라는 뜻입니다(11-04).

$$\text{loss} = \sum_{m,\,n=1}^{V} f(X_{mn}) \left(w_m^\top \tilde{w}_n + b_m + \tilde{b}_n - \log X_{mn} \right)^2, \qquad f(x) = \min\left(1,\ (x/x_{\max})^{3/4}\right)$$

괄호 안은 "모델이 낸 값 - 실제로 센 값"이고, 03장의 평균 제곱 오차처럼 제곱해서 더합니다. f는 단어 쌍마다의 가중치입니다.

왜 가중치 f를 곱할까? 동시 등장 행렬은 대부분의 칸이 0이거나 아주 작은 값입니다. 한두 번 함께 나온 쌍은 우연일 수 있으므로 낮은 가중치를 줍니다. 반대로 "it is"처럼 지나치게 자주 나오는 쌍이 학습을 지배하지 않도록 가중치의 최댓값을 1로 막아 둡니다. 11-03에서 TF-IDF가 흔한 단어의 영향을 줄인 것과 같은 고민입니다.

예시 적용 세 문장 "I like deep learning", "I like NLP", "I enjoy flying"으로 윈도우 크기 1의 동시 등장 행렬을 만듭니다(실습 7).

동시 등장 행렬 X (7, 7): 행의 단어 바로 옆에 열의 단어가 나온 횟수

	I	like	enjoy	deep	learning	NLP	flying	
I	0	2	1	0	0	0	0	I like 가 두 번 나옴
like	2	0	0	1	0	1	0	
enjoy	1	0	0	0	0	0	1	
deep	0	1	0	0	1	0	0	
learning	0	0	0	1	0	0	0	
NLP	0	1	0	0	0	0	0	
flying	0	0	1	0	0	0	0	

대각선을 기준으로 위아래가 같다 (대칭). 49칸 중 37칸이 0이다

그림 7 "I like"가 두 문장에 나와 I행 like열과 like행 I열이 2다. 대부분의 칸이 0이다

like 행의 합은 4입니다. 그래서 $P(\text{I} \mid \text{like}) = 2/4 = 0.50$, $P(\text{deep} \mid \text{like}) = 1/4 = 0.25$, $P(\text{NLP} \mid \text{like}) = 1/4 = 0.25$ 입니다.

GloVe 논문이 든 예는 동시 등장 확률의 **비**가 단어의 관계를 드러낸다는 것입니다.

	k = solid (단단한)	k = gas (기체)	k = water (물)	k = fashion (패션)
$P(k \mid \text{ice})$	0.00019	0.000066	0.003	0.000017
$P(k \mid \text{steam})$	0.000022	0.00078	0.0022	0.000018
비 (위 ÷ 아래)	8.9 (1보다 훨씬 큼)	0.085 (1보다 훨씬 작음)	1.36 (1에 가까움)	0.96 (1에 가까움)

solid는 얼음(ice)과 관련이 깊어 비가 크고, gas는 증기(steam)와 관련이 깊어 비가 작습니다. water는 둘 다와 관련 있고 fashion은 둘 다와 관련 없어 1에 가깝습니다. GloVe는 이런 통계가 벡터에 담기도록 학습합니다.

사전 훈련된 GloVe 50차원 벡터(단어 40만 개)로 유추를 해 보면, king - man + woman에 가장 가까운 단어는 queen(0.85), seoul - korea + japan에 가장 가까운 단어는 tokyo(0.94)입니다.

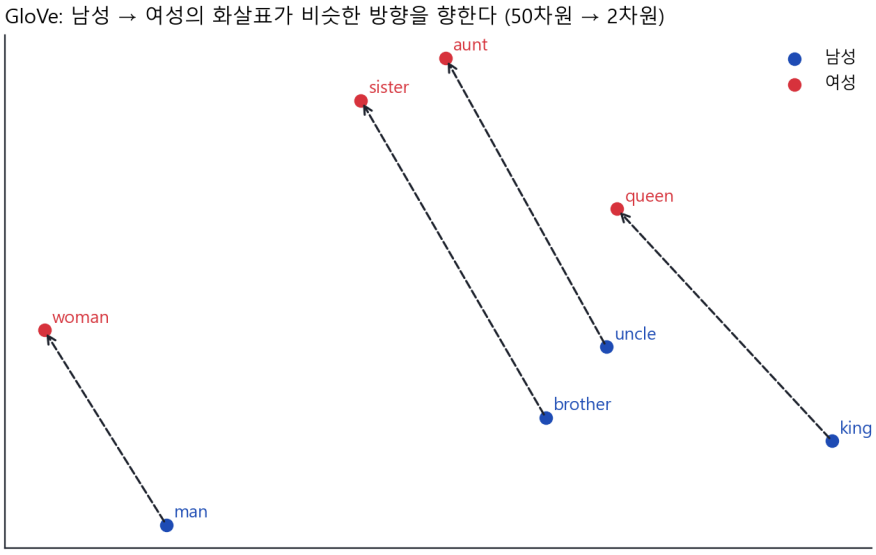


그림 8 GloVe 벡터를 2차원으로 줄인 그림. man → woman, brother → sister, uncle → aunt, king → queen의 화살표가 비슷한 방향을 향한다. "성별"이라는 관계가 벡터의 방향으로 담겼다

2) FastText: 단어를 글자 조각으로

정의

FastText는 페이스북에서 만든 방법으로 Word2Vec의 확장입니다. Word2Vec은 단어를 더 쪼갤 수 없는 단위로 보지만, FastText는 **단어 안에 더 작은 단위(내부 단어, subword)가 있다**고 봅니다.

- 단어의 앞뒤에 시작과 끝을 뜻하는 `<`, `>`를 붙이고, 글자 n개씩 잘라 **글자 n-gram**을 만듭니다. 보통 n은 3부터 6까지 모두 씁니다.
- 단어 전체에 `<`, `>`를 붙인 것도 하나의 조각으로 넣습니다.
- 조각마다 벡터를 학습하고(Word2Vec과 같은 방법), **단어의 벡터는 그 조각들의 벡터를 모두 더한 것**으로 정합니다.

이렇게 하면 Word2Vec에 없던 장점이 생깁니다.

- **모르는 단어(OOV)에 대응한다.** 학습할 때 없던 단어라도, 그 단어의 조각들이 다른 단어에 있었다면 조각의 벡터를 더해 벡터를 만들 수 있습니다. birthplace를 본 적이 없어도 birth와 place의 조각으로 만들 수 있습니다. Word2Vec과 GloVe는 모르는 단어에 벡터를 줄 수 없습니다.
- **드문 단어와 오타에 강하다.** 드물게 나온 단어는 주변 단어의 예가 적어 Word2Vec으로는 벡터를 잘 배우지 못합니다. FastText는 조각을 다른 단어와 나눠 쓰므로 조각의 벡터는 충분히 학습됩니다. 오타가 섞인 단어도 올바른 단어와 조각을 많이 함께 가져 비슷한 벡터가 됩니다.
- **한국어**에서는 음절 단위로 자르거나("자연어처리" → <자연, 자연어, 연어처, 어처리, 처리>), 초성·중성·종성의 자모 단위로 풀어서 자르는 방법을 씁니다. 자모 단위가 오타에 더 강합니다.

예시 적용

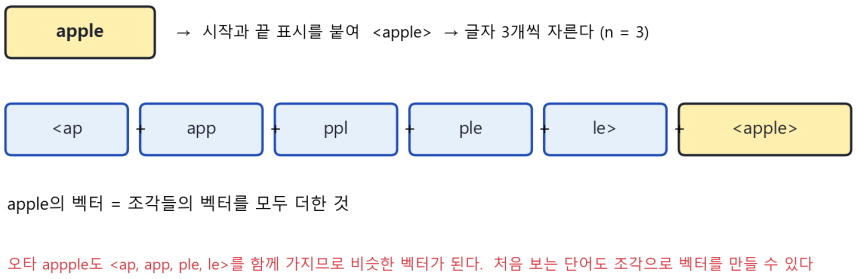


그림 9 apple을 글자 3개씩 자른 조각 5개와 단어 전체. apple의 벡터는 이 조각들의 벡터의 합이다

n을 3부터 6까지 쓰면 apple의 조각은 15개입니다. 오타 appple은 그중 9개를 함께 가집니다(실습 8). 그래서 두 단어의 벡터가 비슷해집니다.

같은 글(text8의 앞 300만 단어)로 Word2Vec과 FastText를 학습한 뒤, 글에 없던 단어 electrofishing(전기로 물고기를 잡는 방법)을 넣어 봤습니다.

모델	electrofishing과 비슷한 단어
Word2Vec	오류: `KeyError "Key 'electrofishing' not present in vocabulary"`
FastText	hydroelectric, electrolytic, electrode, electrodes, electrolyte

FastText는 처음 보는 단어에도 답을 냈습니다. 다만 결과가 모두 electr-이라는 글자를 가진 단어이고, fishing 쪽 뜻은 반영되지 않았습니다. FastText가 주는 것은 **글자가 비슷한 단어와 가까운 벡터**이지 뜻을 이해한 벡터는 아닙니다.

3) 세 방법의 비교

정의

	Word2Vec	GloVe	FastText
학습에 쓰는 정보	윈도우 안의 주변 단어 (예측)	말뭉치 전체의 동시 등장 횟수 (카운트 + 예측)	윈도우 안의 주변 단어 + 글자 n-gram
벡터를 주는 단위	단어	단어	글자 조각 (단어 = 조각의 합)
모르는 단어 (OOV)	벡터 없음	벡터 없음	조각으로 벡터를 만듦
오타, 드문 단어	약함	약함	강함
만든 곳, 연도	구글, 2013	스탠퍼드, 2014	페이스북, 2016

어느 하나가 항상 낫지는 않습니다. 보통 여러 방법을 시도해 보고 하려는 일에서 성능이 좋은 것을 고릅니다.

예시 적용

영화 리뷰처럼 오타와 줄임말이 많은 글("잼", "젬", "재밋")이라면 FastText가 유리합니다. 실습 6에서 "재미"와 비슷한 단어로 "젬"이 나온 것은 우연히 비슷한 자리에 쓰였기 때문이고, FastText라면 글자 조각이 겹쳐서 더 안정적으로 가깝게 나옵니다. 뉴스 기사처럼 정제된 글에서 단어 사이의 관계를 보려면 GloVe나 Word2Vec의 사전 훈련된 벡터를 가져다 쓰는 것이 간편합니다.

비유

처음 보는 한자어의 뜻 짐작하기

"수목원"이라는 말을 처음 봐도 "수(樹, 나무)", "목(木, 나무)", "원(園, 동산)"을 알면 나무가 많은 곳이라고 짐작할 수 있습니다. 단어를 통째로만 외운 사람은 처음 보는 단어 앞에서 멈추지만, 글자의 뜻을 아는 사람은 조각으로 짐작합니다. 물론 조각만으로 짐작하면 틀릴 때도 있습니다.

단어를 통째로 외움 = = Word2Vec, GloVe
글자의 뜻으로 짐작 = = FastText (조각의 합)
처음 보는 단어 = = OOV

실습 7

동시 등장 행렬, 그리고 사전 훈련된 GloVe 벡터

```
import gensim.downloader as api
import numpy as np

# 1) 윈도우 기반 동시 등장 행렬 (윈도우 크기 1: 바로 옆 단어만 센다)
corpus = ["I like deep learning", "I like NLP", "I enjoy flying"]
vocab = ["I", "like", "enjoy", "deep", "learning", "NLP", "flying"]
w2i = {w: i for i, w in enumerate(vocab)}
X = np.zeros((len(vocab), len(vocab)), dtype=int)
for sentence in corpus:
    tokens = sentence.split()
    for a, b in zip(tokens, tokens[1:]):          # 이웃한 두 단어
        X[w2i[a], w2i[b]] += 1
        X[w2i[b], w2i[a]] += 1                    # 반대 방향도 같은 값 (대칭)
print("동시 등장 행렬 X:", X.shape)
print("          " + "".join(f"{w:>9}" for w in vocab))
for w in vocab:
    print(f"{w:>9}" + "".join(f"{v:>9}" for v in X[w2i[w]]))

# 2) 동시 등장 확률 P(k | i) = X[i, k] / (i행의 합)
i = w2i["like"]
print("\n'like' 행의 합:", X[i].sum())
for k in ["I", "deep", "NLP"]:
    count, total = X[i, w2i[k]], X[i].sum()
    print(f"  P({k} | like) = {count}/{total} = {count / total:.2f}")

# 3) 사전 훈련된 GloVe 벡터로 확인 (위키백과 등 60억 단어로 학습, 50차원)
```

```
glove = api.load("glove-wiki-gigaword-50")
print("\nGloVe 임베딩 행렬:", glove.vectors.shape)
for a, b in [("ice", "water"), ("ice", "steam"), ("ice", "fashion")]:
    print(f"  유사도 {a}-{b}: {glove.similarity(a, b):.2f}")

# 4) 유추: king - man + woman, seoul - korea + japan
for pos, neg in [(["king", "woman"], ["man"]), (["seoul", "japan"], ["korea"])]:
    answer = glove.most_similar(positive=pos, negative=neg, topn=1)[0]
    print(f"  {pos[0]} - {neg[0]} + {pos[1]} ≈ {answer[0]} ({answer[1]:.2f})")
```

실행 결과

```
동시 등장 행렬 X: (7, 7)
                I     like    enjoy     deep learning      NLP   flying
        I       0        2        1        0        0        0        0
     like       2        0        0        1        0        1        0
    enjoy       1        0        0        0        0        0        1
     deep       0        1        0        0        1        0        0
 learning       0        0        0        1        0        0        0
      NLP       0        1        0        0        0        0        0
   flying       0        0        1        0        0        0        0

'like' 행의 합: 4
  P(I | like) = 2/4 = 0.50
  P(deep | like) = 1/4 = 0.25
  P(NLP | like) = 1/4 = 0.25

GloVe 임베딩 행렬: (400000, 50)
  유사도 ice-water: 0.68
  유사도 ice-steam: 0.47
  유사도 ice-fashion: 0.30
  king - man + woman ≈ queen (0.85)
  seoul - korea + japan ≈ tokyo (0.94)
```

직접 해 보기 유추를 더 해 보세요: `positive=["paris", "germany"], negative=["france"]` (paris - france + germany). 어떤 단어가 나올까요?

실습 8 글자 n-gram, 그리고 Word2Vec과 FastText 비교

```
import gzip

import gensim.downloader as api
from gensim.models import FastText, Word2Vec

# 1) 단어를 글자 n-gram으로 쪼개기
def char_ngrams(word, n_min=3, n_max=6):
    w = f"<{word}>"                              # 시작 <, 끝 > 표시
    grams = [w[i:i + n] for n in range(n_min, n_max + 1)
             for i in range(len(w) - n + 1)]
    return grams + [w]                           # 단어 전체도 하나로 넣는다

print("apple, n = 3   :", char_ngrams("apple", 3, 3))
print("apple, n = 3~6 :", len(char_ngrams("apple")), "개")
a, b = set(char_ngrams("apple")), set(char_ngrams("appple"))      # 오타 appple
print("apple과 appple이 함께 가진 n-gram:", len(a & b), "개,",
      "apple 전체:", len(a), "개")

# 2) 같은 글로 Word2Vec과 FastText를 학습 (text8의 앞 300만 단어, 1~2분)
path = api.load("text8", return_path=True)
words = gzip.open(path, "rt", encoding="utf8").read().split()[:3_000_000]
sentences = [words[i:i + 1000] for i in range(0, len(words), 1000)]
w2v = Word2Vec(sentences, vector_size=100, window=5, min_count=5, workers=4, sg=1)
ft = FastText(sentences, vector_size=100, window=5, min_count=5, workers=4, sg=1)

# 3) 학습 글에 없던 단어: electrofishing
word = "electrofishing"
print(f"\n'{word}'가 단어 집합에 있나?", word in w2v.wv.key_to_index)
try:
    w2v.wv.most_similar(word)
except KeyError as e:
    print("Word2Vec:", "KeyError", e)
print("FastText:", [w for w, _ in ft.wv.most_similar(word, topn=5)])
```

실행 결과

```
apple, n = 3   : ['<ap', 'app', 'ppl', 'ple', 'le>', '<apple>']
apple, n = 3~6 : 15 개
apple과 appple이 함께 가진 n-gram: 9 개, apple 전체: 15 개

'electrofishing'가 단어 집합에 있나? False
Word2Vec: KeyError "Key 'electrofishing' not present in vocabulary"
FastText: ['hydroelectric', 'electrolytic', 'electrode', 'electrodes', 'electrolyte']
```

직접 해 보기 `word`를 오타인 "goverment"(n이 하나 빠짐)로 바꿔 보세요. FastText는 government와 비슷한 단어를 찾아 주나요?

확인 문제

12-05 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 GloVe는 말뭉치 전체에서 센 동시 등장 횟수를 학습에 쓴다. (O / X) 이름 _____

2 Word2Vec은 학습할 때 보지 못한 단어에도 벡터를 만들어 준다. (O / X) 이름 _____

3 FastText에서 단어의 벡터는 어떻게 만드나요? 이름 _____

① 글자 n-gram 조각들의 벡터를 더한다 ② 원-핫 벡터를 그대로 쓴다 ③ 문서의 평균을 낸다

4 동시 등장 행렬이 대칭인 이유는? 이름 _____

① i 옆에 k가 나온 횟수와 k 옆에 i가 나온 횟수가 같아서 ② 단어를 정렬해서 ③ 로그를 씌워서

5 실습에서 $P(\mathrm{I} \mid \mathrm{like})$는 2/4 = _____이었다. 이름 _____

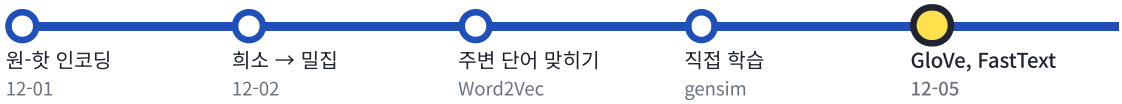
6 apple을 n = 3으로 자르면 <ap, app, _____, ple, le>가 된다. 이름 _____

7 Word2Vec, GloVe, FastText의 차이를 "학습에 쓰는 정보"와 "모르는 단어" 두 가지로 설명해 보세요. 이름 _____

원문: 12-05 글로브와 패스트텍스트 — https://wikidocs.net/60858 (교재의 GloVe 손실 함수 유도 과정은 결과 식과 뜻만 실었습니다. 유도는 원문을 참고하세요)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
원-핫 벡터	길이 V, 번호 자리만 1. 유사도는 항상 0	자연어 = [0, 0, 1, 0, 0, 0]
밀집 벡터, 워드 임베딩	길이를 정할 수 있고 값이 실수인 벡터 / 단어를 그렇게 나타내는 방법	dog-cat 유사도 0.92 (원-핫은 0.00)
분포 가설	비슷한 위치에 나오는 단어는 뜻이 비슷하다	강아지, 고양이 근처의 "귀엽다"
CBOW / Skip-gram	주변 단어 → 중심 단어 / 중심 단어 → 주변 단어	[fat, cat, on, the] → sat
룩업, 투사층	원-핫 × W = W의 한 행 / 꺼낸 벡터들의 평균	(4, 5) → (5,) → (7,)
네거티브 샘플링	V개 중 고르기를 O/X 문제 몇 개로 바꿔 빠르게	71,290개 → 6개
gensim Word2Vec	토큰 리스트 → 임베딩 행렬 (V, M)	(71290, 100). king - man + woman ≈ queen
GloVe	내적이 동시 등장 횟수의 로그가 되도록 학습	seoul - korea + japan ≈ tokyo
FastText	단어 = 글자 n-gram 조각들의 벡터의 합	electrofishing에도 답을 냄

다음

다음 자료 예고: 12장 뒷부분 (임베딩을 신경망에서 쓰기)

임베딩 벡터를 만드는 방법을 배웠으니, 이제 신경망의 입력으로 쓰는 방법을 봅니다. 파이토치의 `nn.Embedding`이 오늘 본 룩업 테이블 W와 같은 것이라는 점, 직접 학습하는 것과 사전 훈련된 벡터를 가져오는 것의 차이, 문맥에 따라 벡터가 달라지는 ELMo, 그리고 임베딩을 넣은 단어 단위 RNN을 다룹니다.

정답과 해설

12-01 NLP에서의 원-핫 인코딩

1. O
2. X — 서로 다른 두 원-핫 벡터의 내적은 항상 0입니다.
3. ①
4. ②
5. 0
6. 2
7. 예시 답: 원-핫 인코딩은 단어 집합 크기의 벡터에서 그 단어의 번호 자리만 1로 두는 표현이다. 단어 사이의 유사도를 나타내지 못하므로, "삿포로 숙소"를 검색해도 숙소와 비슷한 말인 호텔, 료칸이 들어간 결과를 함께 보여 줄 수 없다.

12-02 워드 임베딩

1. O
2. X — 사용자가 정합니다. 단어 40만 개를 50칸으로 나타냈습니다.
3. ②
4. ①
5. 희소
6. 0.92
7. 예시 답: 희소 표현은 값 대부분이 0인 표현, 밀집 표현은 길이를 정할 수 있고 모든 칸이 실수인 표현이다. 색에 번호를 붙이면 번호가 가까워도 색이 비슷한지 알 수 없지만, RGB 세 숫자로 나타내면 숫자가 비슷할 때 색도 비슷하다. 임베딩 벡터가 RGB에 해당한다.

12-03 워드투벡터(Word2Vec)

1. O
2. X — 은닉층이 투사층 하나뿐인 얕은 신경망이고 활성화 함수도 없습니다.
3. ②
4. ①
5. 분포
6. 4 — 앞 2개, 뒤 2개
7. 예시 답: 주변 단어의 원-핫 벡터(길이 V)에 W를 곱하면 W의 한 행(길이 M)이 나온다(룩업). 주변 단어들의 벡터를 평균 내 길이 M인 벡터 하나로 만든다. 여기에 W'를 곱해 길이 V인 점수를 얻고, 소프트맥스로 합이 1인 확률로 바꾼다. 예문에서는 (4, 5) → (5,) → (7,)이었다.

12-04 Word2Vec 학습 및 시각화

1. O
2. X — 글에서 주변 단어를 맞히는 문제만으로 학습합니다.
3. ②
4. ②
5. queen
6. 차원
7. 예시 답: Word2Vec은 비슷한 자리에 나오는 단어를 가깝게 만든다. 데이터가 3만 개로 적어서, 뜻이 비슷한 말 대신 "근래 최고", "단연 최고"처럼 최고 옆에 자주 붙어 다니는 말이 가깝게 나왔다. 데이터가 많아야 품질이 좋아진다.

12-05 글로브와 패스트텍스트

1. O
2. X — KeyError가 납니다. 조각으로 벡터를 만드는 것은 FastText입니다.
3. ①

4. ①
5. 0.5 (0.50)
6. ppl
7. 예시 답: Word2Vec은 윈도우 안의 주변 단어만 보고, GloVe는 말뭉치 전체의 동시 등장 횟수를 쓰며, FastText는 주변 단어에 더해 글자 n-gram 조각을 쓴다. Word2Vec과 GloVe는 모르는 단어에 벡터를 줄 수 없지만, FastText는 조각의 벡터를 더해 만들 수 있다.

용어 정리

용어	정의
원-핫 인코딩	길이 V인 벡터에서 단어의 번호 자리만 1로 두는 표현 방법
희소 표현 / 밀집 표현	값 대부분이 0인 표현 / 길이를 정할 수 있고 모든 칸이 실수인 표현
워드 임베딩, 임베딩 벡터	단어를 밀집 벡터로 나타내는 방법 / 그렇게 얻은 벡터
분포 가설	비슷한 위치에 등장하는 단어는 비슷한 뜻을 가진다는 가정
중심 단어 / 주변 단어 / 윈도우	지금 보는 단어 / 그 앞뒤의 단어 / 앞뒤로 볼 단어 수
CBOW / Skip-gram	주변 단어로 중심 단어 맞히기 / 중심 단어로 주변 단어 맞히기
룩업, 룩업 테이블	번호로 가중치 행렬의 한 행을 꺼내는 것 / 그 행렬
투사층	룩업한 벡터들의 평균을 내는 층. 활성화 함수가 없음
네거티브 샘플링	전체 단어 대신 정답과 무작위로 뽑은 몇 개만 계산하는 방법
gensim	Word2Vec, FastText 등을 제공하는 파이썬 패키지
사전 훈련된 워드 임베딩	방대한 글로 미리 학습해 공개된 임베딩 벡터
동시 등장 행렬	단어 i의 윈도우 안에 단어 k가 나온 횟수를 i행 k열에 적은 행렬
내부 단어 (subword), OOV	단어를 쪼갠 글자 조각 / 단어 집합에 없는 단어